CL-2001
Data Structures
Lab # 8

Objectives:

1. Recursion
2. Binary Trees

Note: Carefully read the following instructions (*Each instruction contains a weightage*)

1. There must be a block of comments at start of every question's code by students; the block should contain brief description about functionality of code.
2. Comment on every function about its functionality.
3. Use understandable name of variables.
4. Proper indentation of code is essential.
5. Make separate .cpp files for all tasks and use this format **22F-1234_Task1.cpp.**
6. First think about statement problems and then write/draw your logic on copy.
7. After copy pencil work, code the problem statement on C++ compiler.
8. Make a Microsoft Word file and paste all of your C++ code with all possible screenshots of every **task output in MS word and submit .cpp file with word file**.
9. Please submit your file in this format **22F-1234_L1**.
10. Do not submit your assignment **after the deadline**.
11. **Do not copy code from any source otherwise you will be penalized with negative marks.**

Problem 1:

You are working on a project that involves navigating through a maze represented as a 2D grid. The maze consists of cells, some of which are blocked walls, while others are open paths.
Your task is to write a program that uses recursion to find a path from the starting position to the exit of the maze.

Write a C++ program that uses **recursion** to solve a maze represented as a 2D grid. The maze will be provided as a 2D array, where `0` represents an open path, and `1` represents a blocked wall. The starting position will be at the top-left corner (0, 0), and the exit will be at the bottom-right corner.

Your program should:

1. Define a recursive function `solveMaze` that takes the maze, the current position `(row, col)`, and the dimensions of the maze as input. This function should return `true` if a path is found, and `false` otherwise.
2. The `solveMaze` function should use recursion to explore all possible paths. It should mark visited cells to avoid cycles.
3. In the `main` function, prompt the user to enter the dimensions of the maze and provide the maze as input.
4. Call the `solveMaze` function to find a path through the maze.
5. If a path is found, display the maze with the path marked. If no path is found, indicate that no solution exists.

Ensure that your program handles edge cases appropriately.

Example Output:

Enter the dimensions of the maze (rows columns): 4 4
Enter the maze (0 for open path, 1 for blocked wall):
0 1 0 0
0 1 0 1
0 0 0 0
0 1 1 0

Solution Found:
1 0 0 0
1 0 0 1
1 1 1 0
0 1 1 0

Problem 2:

Consider the following binary tree:

```
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};
TreeNode* buildTree() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    return root;
}
```

Write a C++ program that performs **inorder**, **preorder**, and **postorder** traversals on the provided binary tree.

Your program should:

1. Implement functions `inorderTraversal`, `preorderTraversal`, and `postorderTraversal` to perform the respective traversals.
2. The functions should take the root of the tree as input and return a vector of integers representing the values of the nodes in the specified order.
3. In the `main` function, use the `buildTree` function to create the binary tree and demonstrate the use of the traversal functions.